Efficient Thinking: Resource Allocation in Game-Playing AI

Spending Parameters, Data, Search, and Latency Efficiently for Maximum Playing Strength — an Empirical Framework, Demonstrated in Chess

Louay Alsakka · July 8, 2026

A white paper on parameter-efficient game AI.

Code & trained model: github.com/louayalsakka/efficient-thinking

The recurring theme is a set of tradeoffs: memory vs compute (Stage 1 vs 2), speed vs score (the search cascade), and diversity vs correlation (the committee) — each one a different way of spending a fixed budget to buy strength, and each capped by the same wall: the quality of the learned evaluator.

Abstract

This paper asks one engineering question: given finite resources — parameters (memory), training data, inference-time search, and latency — how do you spend them *efficiently* to reach a target playing strength? We treat chess as a clean, fully-measurable testbed and map the tradeoff curves directly, resource by resource. With a deliberately tiny model on modest hardware, a **14 MB** convolutional evaluator (3.45M params) plays at ~2150 Elo as a single forward pass; adding **Monte-Carlo Tree Search (MCTS)** lifts the *same weights* to **~2800** with zero extra parameters (**+286 Elo over the raw policy from pure inference compute**, same-ladder) — strength bought with **compute, not size**. **The ~2800 target is deliberate, not a ceiling we failed to exceed:** ~2800 is Super-GM / peak-human level, and our question is *how small a model can match top-human capability when augmented by search* — the human-peak band (which ~2800 ±100 brackets) is the intended saturation point, not a race against 3500-Elo engines. That single result is the thesis in miniature: the same capability can be purchased with very different resource mixes, and the efficient mix is rarely "more parameters". *(If you remember three numbers, remember these: a 14 MB evaluator, +286 Elo from search alone (→ ~2800, human-peak), and 4.8× search efficiency from the cascade.)*

The primary contribution is an empirical *framework* for allocating finite AI resources — parameters, data, search, latency — to a fixed capability goal, with measured tradeoff curves; the staged MCTS cascade is its sharpest demonstration. Three results support it. **(1)** A **wide→narrow MCTS cascade** matching flat MCTS at up to **~4.8× less compute per move** — our most practical result. **(2)** A direct **MCTS-vs-fixed-depth** comparison: adaptive search beats alpha-beta at equal compute and *keeps scaling*, while fixed depth plateaus. **(3)** A **teacher-free self-learning** study: one robust positive — **agreement predicts correctness** — and honest negatives — self-play, a self-referential ladder, evolution, and plurality voting all fail to cross the plateau or de-bias. A **capacity sweep** agrees: 1.4× more parameters at fixed data (3.45M→4.81M) move strength **~0** while 8× more *data* moves it **~+90** — capacity is the *weakest* lever here (a full 1×/1.4×/2×/4× sweep is confirming it). (A small-scale statement: AlphaZero-scale self-play bootstraps far past its start.)

The unifying principle — the **evaluator–search decomposition** — is **strength = evaluator × search**: search sets how *closely* you approach the evaluator's ceiling; the evaluator sets *where that ceiling is* — and that ceiling is **the quality of the information the evaluator was trained on**. Within a move, search *converts* the net's latent information into better decisions (cutting variance, not bias); it cannot create what the net never learned — the reason voting and merging don't help either. **Self-play and evolution differ in kind**: they *can* inject new information from the environment, so at AlphaZero scale they break past human play, but **at two-Mac-Studio compute they plateau — a *scale* limit, not a claim that self-play fails.** The method is as transferable as the numbers: **at each stage one lever binds; only an experiment reveals which, so effort on any other returns almost nothing.**

Headline: *A 14 MB evaluator plus adaptive search reaches ~2800-class play, and a staged MCTS cascade recovers up to 4.8× compute at little Elo cost — thinking, not growing. Search converts what the evaluator already knows into stronger play; it cannot lift the ceiling set by the quality of the information the evaluator was given.***

> ***Read the numbers carefully.*** *Absolute Elo is measured against a Stockfish ladder and carries* ***±~100 systematic uncertainty*** *near the top rung — "~2800" is an efficiency indicator, not an engine-matching claim, and it is the single easiest figure for a skeptic to attack. The* ***robust*** *results are the relative, same-ladder ones: search adds* ***+286 Elo*** *over the raw policy, MCTS beats and out-scales fixed depth, the cascade holds Elo while cutting compute up to 4.8×, and every Stage-3 aggregation/self-learning method fails to beat a single model. Treat those as the paper's claims; treat 2800 as a headline.*
>
> ***Target calibration (design philosophy).*** *We aimed at the* ***human-peak band (~2800, Super-GM)*** *on purpose, not as a failed attempt to beat engines. For human-facing applications, matching top human capability is the efficient saturation point;*

> *pushing a model to 3500 Elo (machine-only territory) spends compute where it stops mattering. The study is therefore "how small a model, plus how little search, reaches human-peak" — **maximum efficiency at the human ceiling**, not a race for absolute strength.*

Established results vs. regime-limited observations. We separate the two explicitly:

- **Established** (robust, relative, same-ladder): the **cascade's compute efficiency**, **adaptive search out-scaling fixed-depth**, and **agreement predicts correctness**. These are same-ladder comparisons that do not lean on absolute Elo.
- **Observations limited to our compute/scale**: the **self-play plateau**, the **flat parameter scaling**, and **evolution's non-escape**. These are true *in our regime* and we expect them to change at AlphaZero/LLM scale — they are honest negatives, not universal laws.

Key Takeaways

Five findings — enough to understand the paper on one page.

1. **Search dominates parameters in this regime.** On a *fixed* evaluator, adding search buys **+286 Elo** over the raw policy and keeps climbing **~+55 per doubling** of simulations; *doubling the parameters at fixed data added ~0*. Strength came from **thinking, not growing**.
2. **A wide→narrow MCTS cascade recovers up to 4.8× compute.** Funnelling the simulation budget through progressively narrower, deeper stages **matches flat-MCTS strength at a fraction of the per-move cost** — our most practical result.
3. **Adaptive MCTS out-scales fixed-depth search.** At equal compute MCTS **beats** alpha-beta and **keeps scaling**, where fixed-depth search plateaus.
4. **Model agreement predicts correctness.** Where independent models agree they are more often right — a **teacher-free confidence signal**, and a result independent of chess.
5. **Find the binding bottleneck experimentally.** At each stage exactly one lever binds — capacity, search, data, or the quality of self-generated signal — and effort on any *other* returns almost nothing. The transferable contribution is this **diagnostic**, not any single Elo number.

---

1. Introduction

Every AI system faces an economic problem. Capability is purchased with finite resources — model capacity (memory), training data, inference-time computation, latency, and human supervision. The engineering objective is not to maximize any one resource but to **maximize capability per unit cost**: more capacity, data, search, or supervision each raise strength, but each carries a cost and to a large degree they *substitute* for one another. This paper presents an **empirical framework for measuring those tradeoffs**, using chess as a clean, fully-observable testbed — we fix the objective (playing strength) and measure what each resource buys, and where spending on one is wasted because a *different* resource is the binding constraint. We claim **no solved optimal-allocation rule**; the contribution is the framework and a set of *measured tradeoff curves*.

The argument is a three-level hierarchy: **(1) AI resource allocation** — the main thesis; **(2) the evaluator–search decomposition** — the analytical model of how capacity and search combine into strength; and **(3) binding-bottleneck analysis** — the diagnostic that identifies, at any stage, which single resource to spend on next.

The experiments are **three allocation questions** of increasing autonomy: 1. **Stage 1 (open loop):** given a fixed memory budget, **what architecture buys the most capability?** 2. **Stage 2 (closed loop):** given a fixed evaluator, **what is the cheapest way to buy additional strength with inference-time computation?** 3. **Stage 3 (self-learning):** without external labels, **what is the most efficient way to improve the evaluator?**

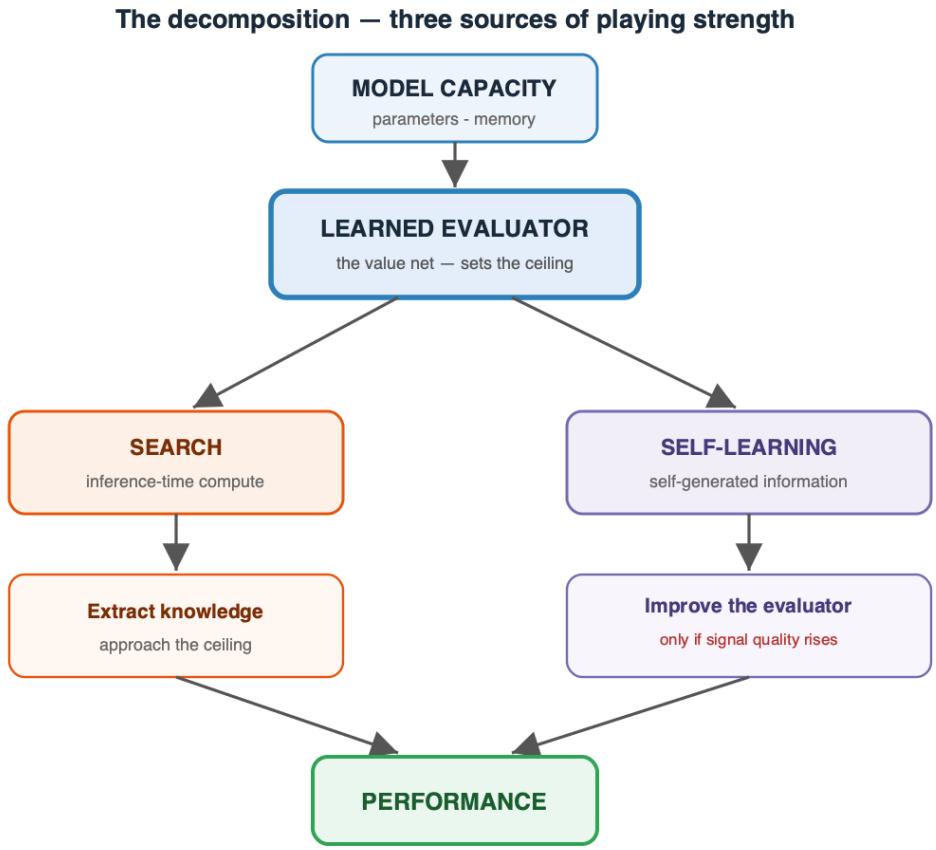
We measure each independently. The framing is also **control-theoretic** (Bertsekas 2022, *Lessons from AlphaZero for Optimal, Model Predictive, and Adaptive Control*): open-loop policy = feedforward controller, closed-loop search = model-predictive control, self-play = iterative/adaptive learning control. Chess is only the testbed; the goal is a *generic* way to reason about resource tradeoffs in sequential decision-making.

The central *tool* for efficient allocation is a simple diagnostic: **at any given stage, strength is gated by a *single binding resource* — capacity, search, data, or the quality of self-generated signal — and spending on any *non-binding* resource returns almost nothing.** So the efficient move is always to **identify the binding resource experimentally, then spend there.** Which one binds is not obvious a priori and shifts as you relieve each (open-loop is capacity-bound; add search and you ride it up log-linearly until the evaluator's quality caps the return; then the evaluator — its *data*, not its parameter count in our regime — binds). This bottleneck analysis is not the paper's goal but its **method**: one instrument in the larger objective of spending a fixed budget efficiently.

Contributions. - **Our headline method — a search-efficiency result:** a wide→narrow MCTS **cascade** that funnels the simulation budget through progressively narrower, deeper stages, matching flat MCTS at up to **4.8× less compute per move** with a clean score/speed trade-off curve. - A parameter-efficiency result: **~2800 Elo from 3.45M params** via

search, at constant memory. - A direct **MCTS-vs-fixed-depth** result: adaptive search beats and out-scales fixed depth. - An **architecture-beats-scale** finding (convolution ≫ MLP at equal data), with topology sweep. - A reproducible **negative result** on small-scale self-play (plateaus below supervision). - A **teacher-free self-learning** study: agreement is a validated **confidence meter**, but self-play, a self-referential ladder, **evolution**, and plurality-voting committees all fail to cross the plateau — a set of clean negative results with a methodological warning (noisy fitness manufactures phantom gains).

The paper's roadmap in one picture — the three sources of strength, all feeding a single learned evaluator:



---

2. Problem Setup and Methods

2.1 Representation

- **Position → move** as a 4096-way (64×64 from–to) classification (promotions default to queen).
- **Side-to-move normalization:** the board is re-oriented so the mover is always "White" (mirror + color-swap for Black), so one network serves both players and the turn needs no bit. (Hence `Eval(P) = Eval(mirror(P))` exactly, and a move's value

reads off as `Eval(B) + Eval(null) − 1`; measured mean tempo ≈ +0.15, up to +0.77 in tactical shots, negative in zugzwang.)
- **Encoding:** *one-hot* = 64×12 piece planes + 5 meta = **773 floats** (reshaped to 8×8×12 + 5 = 17 channels for convolution), used throughout. Against a compact *packed* encoding (one 4-bit code per square, 69 floats) at equal budget and data, one-hot is decisively stronger (**84.9 vs 142.5 mean CPL**): nets learn categorical piece-type features far more readily than an arbitrary ordinal code, and convolution *requires* the per-type planes a packed board cannot provide.

2.2 Labels and objective

- **Supervised data:** the full public Lichess cloud-eval database — **394,669,566 positions** (79 shards), each with Stockfish multi-PV win-probabilities.
- **Hard objective:** cross-entropy to the single best move (bounded by imitation).
- **Soft objective:** cross-entropy to an *advantage-weighted distribution* over legal moves ($\mathrm{softmax}(v_i/\tau)$). Soft is not bounded by copying one move and generalizes better.

2.3 Evaluation

- **Elo via a Stockfish ladder** (random baseline, then SF at set UCI_Elo rungs), 20–80 games/rung; Elo fit by maximum-likelihood against the known rung ratings.
- **Methodological caveat (important):** if the player beats the top rung, the Elo estimate **compresses/extrapolates**, so we raised the ladder as the player improved (SF-1700 → 2100 → 2700 → 3000). Absolute numbers carry systematic uncertainty; **relative gains measured on the same ladder are the robust results** (treat absolutes as ±100). Where two methods are compared, they are always run on the *same* ladder and seeds.

2.4 Hardware

- Two Apple-Silicon **Mac Studios (M3 Ultra, 256 GB each)**, MLX framework, 40 Gbps bridge.

2.5 Reproducibility (evaluation protocol)

A fixed protocol makes every Elo/CPL figure comparable across methods: - **Engine:** Stockfish 18 as ladder opponent and CPL oracle; opponents at fixed `UCI_Elo` rungs, movetime **0.03–0.04 s**; CPL at **fixed depth 12**. - **Ladder:** a random anchor plus `UCI_Elo` rungs (e.g. 1700/2000/2300, 2400/2700/3000), Elo by maximum-likelihood fit; **20–40 games/rung**; margin ≈ $\pm 400/\sqrt{n}\cdot 2$ (**±100** typical) — rely on *relative* same-ladder deltas. - **Openings/seeds:** sampled from the Lichess `2013-01` PGN, replayed to plies 6–16, both colors; seed 0 by default, and any two compared methods use the **same** ladder, openings, and seeds. - **Caveats:** the ladder **compresses** once the player beats the top rung (we raised it as

strength grew), and at 0.03–0.04 s Stockfish plays **below** nominal `UCI_Elo` — internally consistent, not calibrated to over-the-board Elo. Exact scripts are in the repo.

---

3. Stage 1 — Open Loop (single-pass policy)

3.1 Topology sweep — architecture beats scale

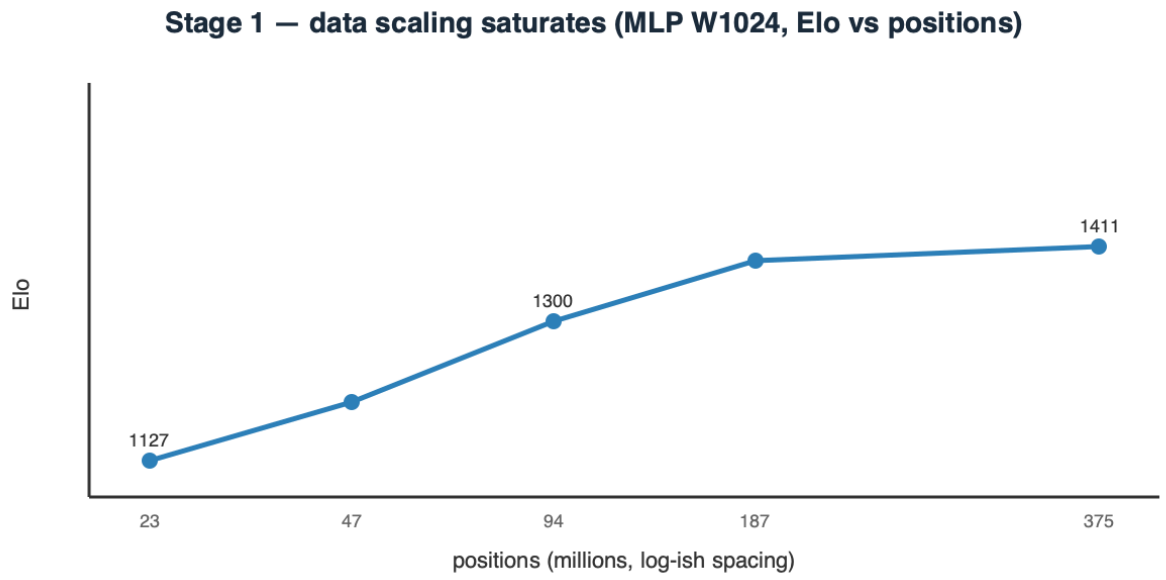
At fixed moderate data (18M positions) we swept eight architectures:

Topology	params	Elo	note
conv (64×10 / 96×8)	2.8–3.4M	**1476**	best & most efficient
constant-width MLP (1024×6)	10.2M	1108–1233	best plain MLP
dual-path (wide+deep, gated)	2.8M	1082	ties MLP at 3.5× fewer params
factored head (from/to)	6.2M	782	−450
funnel (1024→64)	1.8M	582	narrowing hurts
pyramid (64→1024)	5.0M	431	bad
bottleneck (512→8)	0.5M	340	broken

Topology conclusion. Only **constant-width** and **convolution** are competitive; every taper, factoring, or bottleneck loses badly, because it destroys positional information before the head uses it, while convolution **reuses local patterns via weight sharing** (learns a motif once, not per square). The rule: **match the prior to the domain's structure (spatial locality) rather than adding width.** A 3.45M conv matched a 14.4M MLP using **22× less data** — architecture, not parameter count, set the strength.

3.2 Scaling laws (width and data)

- **Width (MLP, hard):** W1024 (14.4M) → 1449; W2048 (47.7M) → 1501: **+52 Elo for 3.3× params** — sharply diminishing. Top-1 accuracy saturates ~0.41 regardless of width.
- **Data (W1024):** 1127 / 1207 / 1300 / 1382 / 1411 at 23 / 47 / 94 / 187 / **375M** positions — large early gains, then **+29 on the last doubling** → saturates near the DB's 394M limit.



3.3 Objective and open-loop ceiling

Soft beats hard by **+94–113 Elo** at subset scale; top-1 saturates while Elo keeps improving via blunder-rate reduction (**top-1 is a misleading metric**). Best open-loop recipe = **conv + soft + full 394M data**, with a **ceiling ≈ 2150 Elo**. Cost: 3.45M params, **14 MB**, **~176 MFLOP / ~1.5 ms per move** — cheap, but capped: no amount of width or data pushed a single pass past ~2150.

---

4. Stage 2 — Closed Loop / Inference-Time Compute (search on a value function)

We add a scalar head **Eval(N) ∈ [0,1]** = *expected score for the side to move* (held-out **MAE 0.088, correlation 0.877** — an excellent value function). Search uses the zero-sum identity — our value after a move is `1 - Eval(child)` — so one network plays both sides; terminals return exact 0 / 0.5 / 1.

4.1 MCTS vs fixed-depth — the core comparison

We compared two search families on the same value net and ladder:

- **Fixed-depth alpha-beta** (negamax, policy move-ordering, quiescence at leaves):

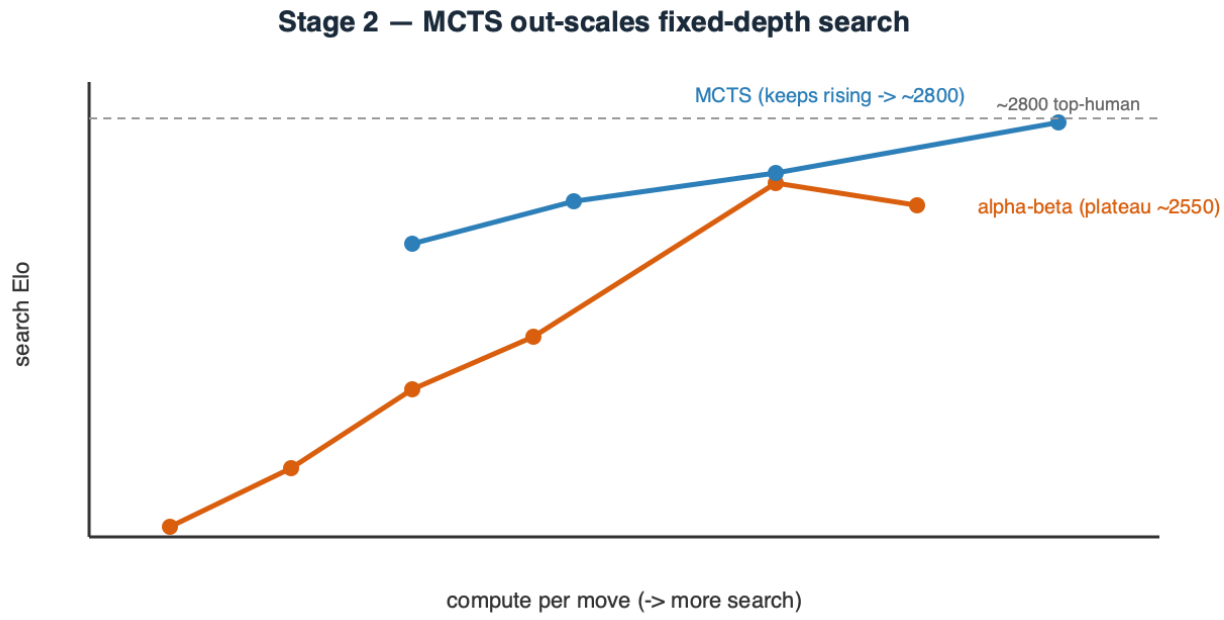
depth	raw	search	gain
1	1661	1627	**−34** (1-ply hurts — can't see the reply)
2	1685	1788	+103

depth	raw	search	gain
3	1895	2005	+110
4	1914	2152	+238
6	2128	**2575**	+447
7	2187	2513	plateau ~2550

• **Adaptive MCTS/PUCT** (`Q + c·P·√N/(1+n)` , value-head leaves, negamax backup):

sims	search Elo
100	2411
200	2530
400	2610 (2661 @ 40 games/rung)
800	**2749 ≈ top-human (~2800)**

Result. (i) **1-ply search *hurts* a strong policy** — it commits to a capture without seeing the recapture; depth is where lookahead pays. (ii) **MCTS beats alpha-beta at equal compute and keeps scaling** where fixed depth plateaus (~2550): uniform depth spends equal effort on every branch and *amplifies the value net's noise*, while MCTS **allocates search adaptively to sharp lines and averages over it.** MCTS wins Stage 2, reaching ~2800 on the fixed 3.45M net.



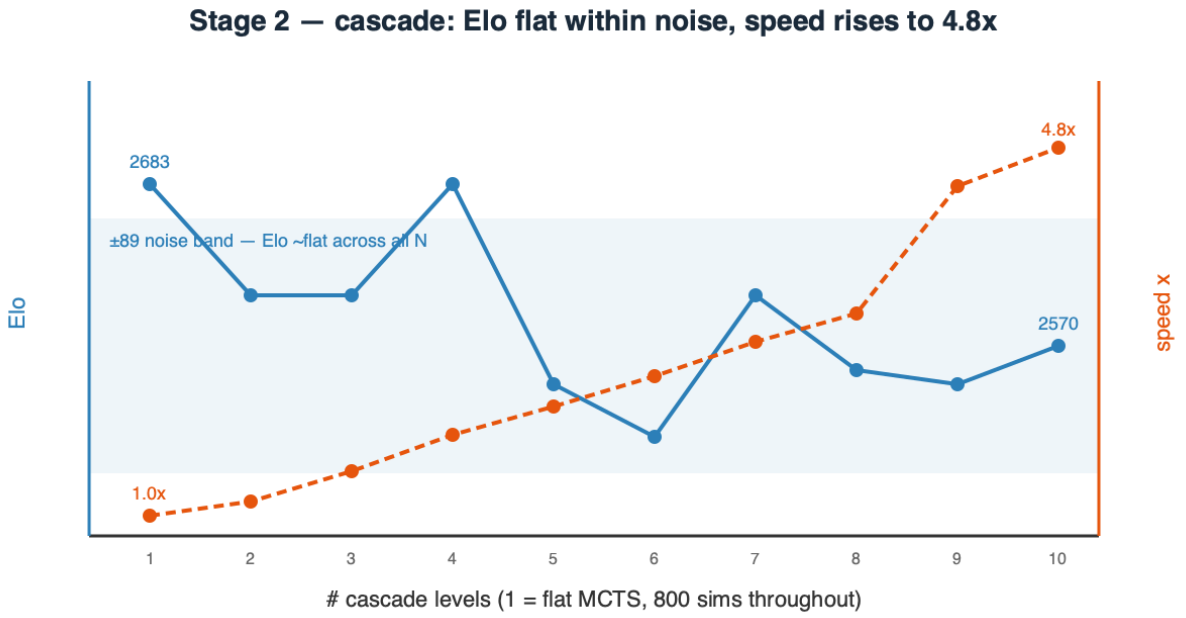
4.2 Search efficiency — the wide→narrow MCTS cascade (new)

Given MCTS wins, does the **allocation** of a fixed sim budget matter? The **cascade** runs MCTS in *stages*: a **wide** stage (all moves, high `c_puct`, few sims) ranks broadly and passes its top-k by visit count to a **narrower, deeper** stage (fewer moves, more sims, lower `c_puct`), funnelling the budget onto survivors; a shared eval cache carries value-net calls forward.

A controlled **N = 1→10 sweep** (one consistent rule generates each funnel; all 800 total sims; same tall ladder, seeds, and openings) gives the full trade-off curve:

# levels	Elo (±89)	ms/move	speedup
1 (flat MCTS)	2683	1330	1.0×
3	2605	903	1.5×
6	2506	541	2.5×
9	2543	300	4.4×
10	2570	275	**4.8×**
beam-minimax cascade (fixed depth)	2487	—	inferior primitive

Result. Across all ten funnels Elo stays within a **single ±89 band** (2506–2683, mean ~2580) — **no significant variation within our ±89 uncertainty** — while speed rises **monotonically to 4.8× at N = 10.** Funnelling wide→narrow is a **near-pure efficiency win**: it holds flat-MCTS strength while cutting per-move compute ~5×, because the survivors reaching the deep stages are the same moves flat MCTS would have searched anyway — the funnel just stops paying for moves it has already ranked out. The practical dial: **more levels = same strength, cheaper.** (The fixed-depth *beam* cascade is a genuinely weaker primitive at 2487 — the adaptive MCTS stages matter.)



4.3 The two-dimensional cost (memory vs GPU-cycles)

	Stage 1 (open)	Stage 2 (closed)
Memory	14 MB	**14 MB — search adds ~0**
GPU cycles/move (batch-1)	1 pass, ~1.5 ms	~800 passes, ~1.3 s (or ~0.6 s cascaded)
GPU cycles/move (batched, §4.4)	—	**~13–37× less — MCTS-3200 below batch-1 MCTS-800**
Elo	~2150 (capped)	**~2800 (scales with compute)**

Stage 1 buys Elo with *memory* and saturates; Stage 2 buys Elo with *GPU cycles* and keeps climbing — and the cascade shows most of those cycles were waste (up to 4.8× recoverable). Note that the ~1.3 s is a **batch-1 implementation artefact**, not the method's cost: batched-leaf evaluation (§4.4, measured 13–37×) makes even MCTS-3200 cheaper than batch-1 MCTS-800, so the true latency axis sits far below what we plot (clean solo-GPU figure pending). The central practical result stands regardless: **strength is compute, not parameters.**

4.4 Search latency — what a second of thinking costs, and buys

Strength from search is not free: every simulation is a neural-network forward pass, so Elo is paid for in wall-clock. Measured on the Mac Studio (M3 Ultra, MLX) across the three operating points:

Mode	Elo (abs. ladder)	Latency / move
Open-loop (raw policy)	~2448	**~2 ms**
MCTS-800	**2734 ±76**	**~1.3 s**
MCTS-1600	**2780 ±82**	~1.9 s
MCTS-3200	**2839 ±76**	~3.8 s
MCTS-6400	**2903 ±82**	~7 s
MCTS-12800	**2967 ±115**	~14 s

All on the same Stockfish high ladder (2500/2800/3050). The round ~2150 / ~2800 headline figures used elsewhere sit within the stated ±100 ladder uncertainty of these precise values.

Three observations:

1. Search scales *log-linearly* — ~+55 Elo per doubling, no saturation through 12800. The first slice is huge and cheap (MCTS-800 alone adds **+286** over the raw policy, 2448→2734); past that, each *doubling* adds a steady **~+55–64 Elo** (2734→2780→2839→2903→2967), unbroken to **16× the base budget** — cumulative **+233**, far above the ±~100 noise. *(An earlier "saturation at 3200" claim came from a noisy head-to-head sweep, +12/+260/+191 per rung; the clean absolute curve supersedes it, and head-to-head deltas inflate via ceiling compression — 3200-vs-800 reads +260 h2h but +105 absolute.)* So with a fixed evaluator search keeps paying, and we **never reach its ceiling** (tested to 16×): marginal Elo *per compute* falls, but the curve does not flatten.

2. Latency scales *sub-linearly* with sims — a red flag, not a feature. MCTS-3200 does 4× the simulations of MCTS-800 yet is only ~2.8× slower. The cause: this search evaluates leaves **one position at a time (batch = 1)**, so each forward pass is dominated by fixed GPU-launch overhead rather than compute — the M3 Ultra is massively under-utilised. Effective throughput is only **~600 leaf-evaluations per second**, and adding sims mostly amortises the fixed per-move cost.

3. The latency is an implementation artefact, not a hardware or method limit. Batched neural-MCTS engines evaluate many leaves per forward pass and reach **~10k–80k nodes/second** (Leela; AlphaZero on TPUs) — 20–100× our batch-1 throughput. We implemented and measured that fix (next), and it changes none of the strength conclusions, only their price.

GPU utilisation — batch-1 is an implementation *limitation*, and we measured the fix. The batch-1 search (obs 2) leaves the M3 Ultra's cores **idle between launches** at only **~600 nps**, so the latencies above are **pessimistic upper bounds**: strength is fixed by **N (nodes searched)**, latency by **N ÷ throughput**, and our throughput is on the floor.

The standard fix is **batched-leaf evaluation** (gather many tree leaves via *virtual loss*, evaluate them in a single GPU launch). We implemented it (`BatchedMCTSPlayer` , `search.py`) and **measured** it on the same net and positions — the same nodes searched, only the launch pattern changed:

Search	Throughput	Speedup vs batch-1
batch-1 (as used above)	~600 nps	1.0×
batched, 32 leaves/launch	—	**13×**
batched, 64 leaves/launch	—	**23×**
batched, 128 leaves/launch	—	**37×**

(ratios measured under concurrent GPU load, so approximate; a clean solo-GPU figure is pending.) A **~13–37×** per-move speedup at identical strength is enough to run **MCTS-3200 well below today's MCTS-800 latency**. **The batch-1 numbers should therefore be read as a ceiling on cost, not the method's efficiency**, and the true strength-vs-latency curve sits far to the left of the one we plot.

Two caveats. First, the gain is **hardware-dependent** — it equals the *idle parallelism you can reclaim*: the wide M3 Ultra leaves much for a 14 MB batch-1 workload, but on a small GPU/CPU or an accelerator already **saturated by a large network** (AlphaZero/Leela), there are no idle cores and deeper search costs full, **linear** price. Second, batched selection uses momentarily stale tree stats, so it is a hair less sample-efficient per node — second-order, not changing the order-of-magnitude speedup.

4.5 Does a bigger evaluator raise the ceiling? (a capacity sweep)

Search still climbs at 12800, but each doubling buys less, so the route higher is a *better evaluator*. We **add parameters** at fixed depth (8) and identical recipe — only width varies.

A correction first. An initial screen mislabelled a width-136 net "2×"; it is in fact **1.4×** (4.81M params), because the policy/value heads scale ~linearly with width, so total parameters grow far slower than the conv body's width². On a fixed **~50M-position** subset:

Net	Params	raw policy	MCTS-800
1× (w96)	3.45M	2298	2631
1.4× (w136)	4.81M	2366	**2649**
Δ	+1.4×	+68	**+18**

+18 Elo with search — not significant within our ±107 uncertainty. For contrast, *8× more data* moves the same architecture **~+90**.

But that screen is confounded — both nets saw only ~50M positions, so a wider net had little extra signal to fill its room. Repeating on the **full ~394M data**, matched to the 2734 baseline:

Capacity (full data)	Params	MCTS-800	Δ vs 1×
1× (w96)	3.45M	2734	—
1.4× (w136)	4.81M	**2794**	**+60**
2× (w184)	7.04M	*training*	—
4× (w288)	14.2M	*training*	—

Already diagnostic: the 1.4×-vs-1× gap **widens from +18 (50M) to +60 (full data)** — within ±110 noise, but the *direction* says the 50M null was **data starvation**, not inert capacity. The **2×/4× points (training)** decide whether the curve keeps climbing or flattens near ~2794. Either way the scoped claim holds: **capacity is the weakest lever *in this data regime***, not "parameters never matter" — at AlphaZero/LLM scale, capacity-bound with abundant data, more parameters clearly help.

The study's lever ranking, all same-ladder:

Lever	Elo moved	Cost
Search (open → 12800 sims)	**+286, then ~+55 / doubling** (log-linear, unsaturated)	×2 latency / doubling
Data (10 shards → full 79)	**~+90**	8× training data
Capacity (1× → 1.4×, at 50M)	**~0** (n.s.; full-data sweep running)	more params & compute

Search dominates, data second, capacity last in this regime — the sharpest reading being the thesis: **one lever binds at each stage; an experiment tells you which. Here it was data and search, not capacity.**

---

5. Stage 3 — Self-Learning (no external engine, no labels)

Why this stage is the whole point. Stages 1–2 leaned on Stockfish labels — a shortcut that exists only because chess already has a superhuman evaluator and a labeled database. The generic goal is the opposite: a **new field with no data and no evaluator** (a novel game, an unsolved control/scheduling problem, a design task), where you can neither imitate a teacher nor score positions with an off-the-shelf engine — you have only the **environment's rules**.

Stage 3 discards every external crutch (no Stockfish, no labels) to test whether strength can be **bootstrapped from self-play and outcomes alone** — the case for any genuinely new problem. The loop: the net plays itself with MCTS, trains toward the visit distribution (policy) and game result (value), and repeats. We studied five approaches — self-play, a self-referential ladder, a committee, evolution, and weight merging — and they converge on one story.

5.1 Self-play expert iteration (AlphaZero-style)

Two failure modes were fixed: **cold-start draw-collapse** (a weak net can't force mates, so games drift to draws and the value head gets no signal — fixed with Dirichlet root noise) and **warm-start forgetting** (hard training on tiny self-play slices overwrote the supervised policy, 2150→1407 — fixed with a replay buffer + gentle LR). Stabilized and parallelized to 16× throughput, the raw policy **plateaus ~1950–2030 — below the ~2150 supervised baseline.**

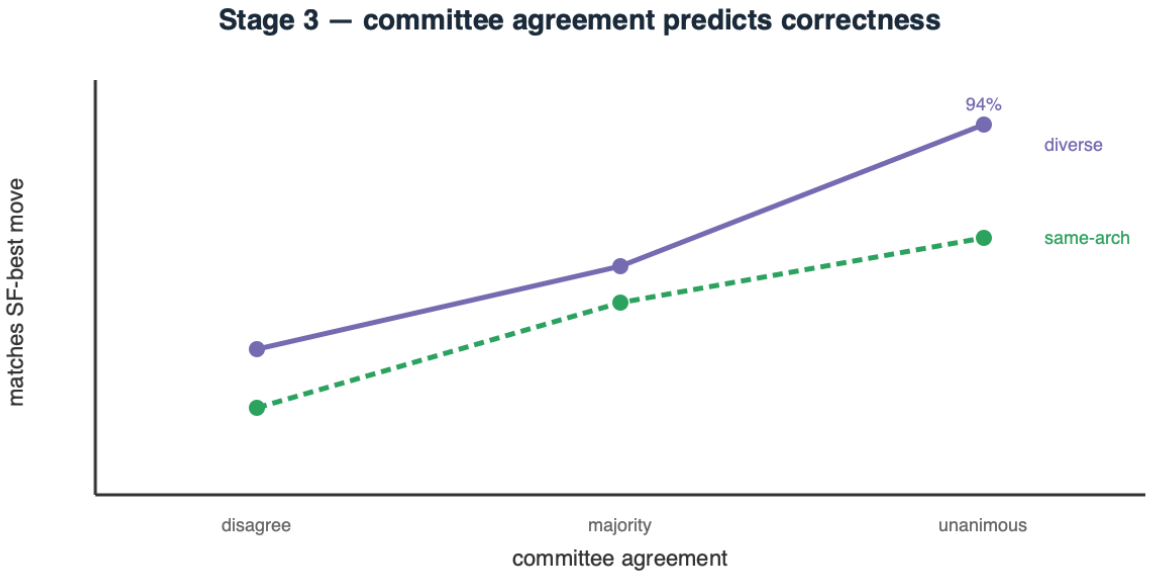
Negative result (clean). At two-machine scale, **self-play converges *below* supervision.** Strength does rise with game volume — a real scaling signal — but the achievable volume plateaus under the 394M-supervised net; crossing it needs orders-of-magnitude more games (AlphaZero used ~1000× ours). **Scale is the binding constraint.**

5.2 Committee / diversity — teacher-free confidence (new)

If the bottleneck is the *evaluator*, the cheap way to improve one without a bigger net is to **ensemble diverse nets**. We tested whether **independently-started models diverge (disagree → uncertain) or converge (agree → likely correct)** — making agreement a confidence meter with no oracle.

Validated. Over 400 positions scored against Stockfish depth-12 (measurement only), agreement strongly predicts correctness:

members agree	centipawn-loss ↓	matches SF-best ↑ (same-arch / diverse)
all disagree	77.9	22% / 37%
majority	40.0	49% / 58%
unanimous	**19.7**	**65% / 94%**



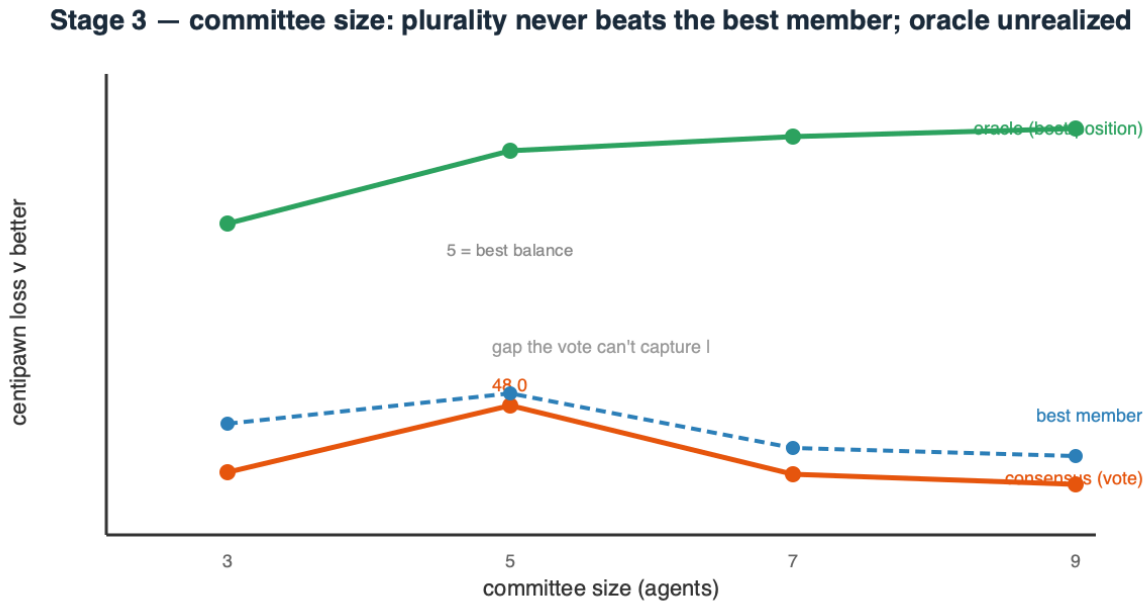
But plurality voting does *not* reliably de-bias — a committee-size sweep (3/5/7/9 agents) shows why. Growing the committee from a correlated conv-soft trio outward:

agents	added views	consensus CPL	best member	oracle
3	conv-soft ×3	54.7	49.8	29.6
5	+MLP +hard-objective	**48.0**	46.8	22.2
7	+2 conv-soft (data slices)	54.8	52.2	20.8
9	+MLP +hard	56.0	53.1	19.9

Three findings. (i) **Plurality never clearly beats the best member** — consensus is slightly *worse* at every size, gaps (~1–5 CPL) inside the ~±3 CPL measurement noise. (ii) **Balance beats count** — the 5-agent committee (diverse MLP + hard-objective = 40% of the vote) is best; *adding correlated members* (conv-soft data-slices at 7, 9) lets that bloc dominate and reverts the gain. **More agents ≠ better.** (iii) **The oracle (best member per position, ~20–30 CPL) is 2–3× better than the vote** — the diversity *contains* the information, but plurality *cannot extract it*, because it is dominated by the largest correlated bloc.

Lesson. The committee robustly gives a **confidence meter** (agreement→correctness) but a **weak aggregator**: plurality does not reliably beat the best member. The large oracle

headroom needs a **better aggregator** (soft-averaging, confidence-weighted routing) and **balanced**, not merely numerous, diversity — a de-biased ensemble is the most promising route to the ceiling search and self-play can't reach, but plurality is not it.



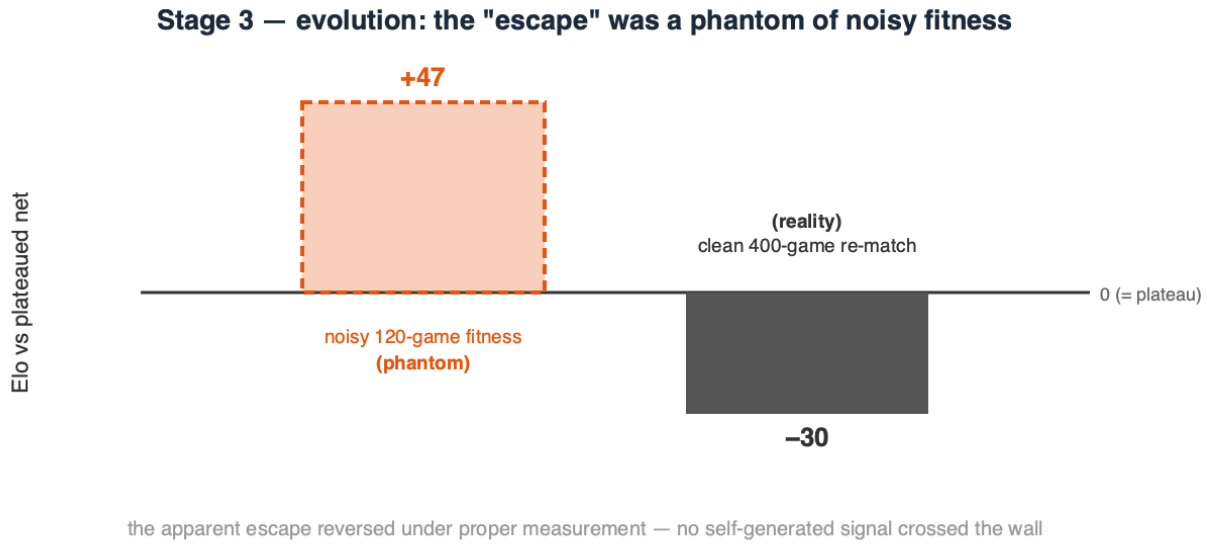
A third combination — averaging *evaluations inside the search* — also fails. Averaging the K models' value at every MCTS leaf, at equal compute (ensemble at 200 sims = 600 passes/move vs a single model at 600 sims), scored **2222 vs 2282, a −60 Elo *loss*:** the ensemble searches 3× less tree, and de-biasing correlated evaluations doesn't justify tripling per-leaf cost. So all three combinations — plurality, weight merging (§5.4), and in-search averaging — fail to beat a single model, for one root cause: **the members' errors are too correlated to cancel.** Genuinely independent evaluators (cross-family, cross-data) are the prerequisite.

5.3 Evolution — mutate / play / score (a plateau-escape attempt)

Gradient self-play optimizes a *proxy* (the net's own biased targets); we tested whether **derivative-free evolution** — optimizing the *true* objective, "did this mutant win games" — could escape the plateau. Each generation: mutate the plateaued net into 16 offspring (Gaussian weight noise), play each against a **frozen copy** (a fixed anchor, so fitness *is* "how well do you beat the plateau"), and crown the best only if it survives a confirmation match. A first version selecting against the *moving* champion drifted **downward** — beating your immediate parent is non-transitive in chess; the fixed anchor fixes that.

Result: a null, and a methodological warning. The run *appeared* to escape — champ-vs-plateau climbed to 0.567 (+47 Elo), passing a 120-game confirmation — but a **clean 400-game, low-temperature re-match found the "evolved" champion is −24 to −41 Elo *worse*.** The gain was an artifact of noisy high-temperature fitness over small samples with best-of-16 selection bias — a **phantom** that vanished under proper measurement; annealing the mutation scale (σ 0.03→0.12) found nothing better. So evolution did not escape either:

neither gradient self-play, a self-referential ladder, nor evolution crosses the ~2000 wall, and since the same net reaches ~2150 under *supervised* labels, the wall is **not capacity** but the ceiling of any *self-generated* signal. (Practitioner note: relative-fitness selection over small stochastic samples manufactures phantom gains — trust only a large, low-variance re-measurement.)



5.4 Model merging — can we *average* diverse models into one?

The weight-space alternative to a voting committee is to **average coefficients into one network**. We tested it on conv-96×8 members from different seeds/data/objectives, scored by mean **CPL** vs Stockfish depth-12 (members ~60 ≈ 2000-level; ~260 ≈ random):

merge	starting points	CPL ↓	verdict
naive average	different-start (diverse)	261	collapse
Git Re-Basin aligned	different-start (diverse)	268	still collapse
naive average (**model soup**)	**same init**	58.8	works (~parent)
aligned (net + permuted twin)	identical	72	perfect recovery *(verification)*

Naive averaging of different-start nets collapses (261): independent nets sit in different loss basins related by neuron permutations, so averaging misaligned neurons cancels signal. The known fix, **permutation alignment (Git Re-Basin)**, we implemented and **verified correct** — it recovers a net *exactly* from a known random permutation (72 CPL). Yet on real different-start members it **still collapses** (268), for a fundamental reason: Git Re-Basin assumes nets learn the *same features in a different order*; ours learned **genuinely different features** (different seeds *and* data *and* objectives), which no permutation aligns. A **model soup** (children from the *same* checkpoint) averages fine (58.8), because a shared start keeps them in one basin.

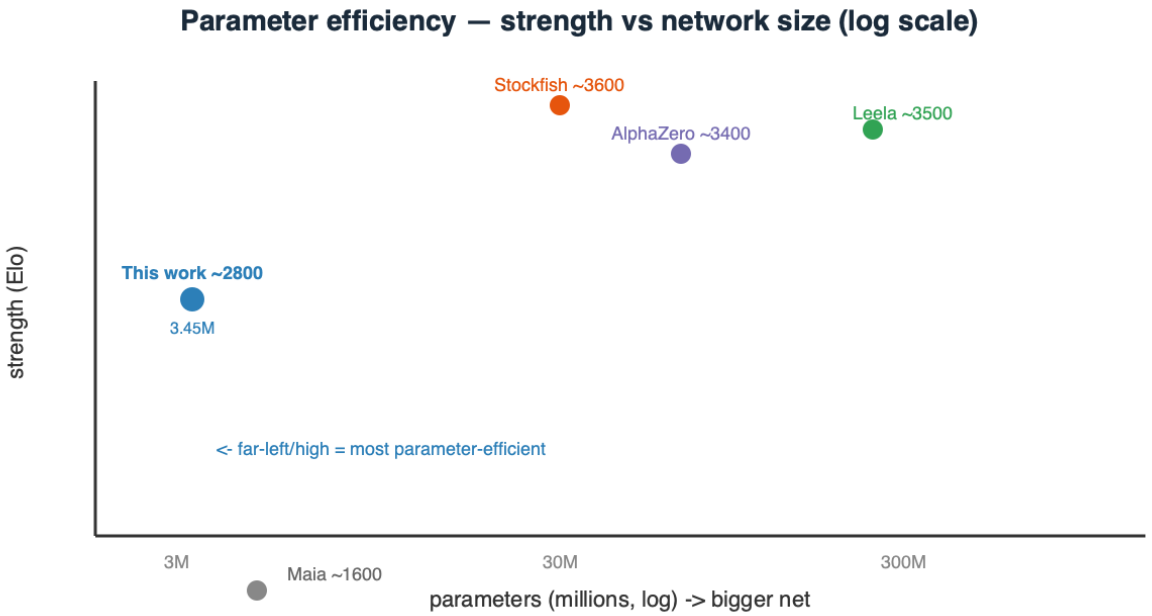
Conclusion. Weight-averaging works only within a shared basin. The very diversity that makes an ensemble valuable (§5.2) is what makes the members' **weights un-averageable** — diverse models combine at *inference*, not in weight space; so the "better aggregator" must live at inference (soft-averaging, confidence routing), not in merging.

---

6. Comparison to existing systems — parameters, performance, and speed

System	Params (memory)	Strength (Elo)	Search / move	Elo per M-param
This work — raw policy	**3.45M** (14 MB)	~2150	1 forward pass (~1.5 ms)	~620
This work — + MCTS-800	**3.45M** (14 MB)	**~2800**	800 net passes (~1.0 s; ~0.6 s cascaded)	**~810**
Maia (human-like)	~few M	~1100–1900	1 forward pass	~300–500
AlphaZero (chess, 2017)	~40–90M	~3400+	800 MCTS sims (big-net passes)	~40–85
Leela Chess Zero (modern)	~100–400M+	~3500+	~1–8k MCTS nodes	~10–35
Stockfish (NNUE)	~tens of M (quantized)	~3600+	**millions of alpha-beta nodes/s**	~100–150

Two axes — size and speed. - **Parameters:** our net is **~10–25× smaller than AlphaZero, 30–100× smaller than large Leela**, yet reaches ~2800 with search — the extreme point on Elo-per-million-parameters. This is an **efficiency framing, not a superiority claim**: top engines are 600–800 Elo stronger and optimize *absolute* strength, not parameters-per-Elo; the point is only that parameter count is *not* what buys their last few hundred Elo (a better value function and far more search are). - **Speed:** AlphaZero/Leela use a similar sim count but each sim is a **big-net** pass (10–100× our network); our sim is a 14 MB pass (~1.5 ms), cascade-recoverable a further ~1.6–4.8×. Our batch-1 search runs at only ~600 nps vs ~10k–80k batched (§4.4). Stockfish is the opposite regime — a tiny quantized net at millions of nodes/s. The structure is identical throughout — a learned **evaluator queried by search** — and **parameter count is not what separates them.**



Honest gap. Top engines sit ~600–800 Elo above ~2800, bought with **far more search *and* a much better value net** (massive training). Our number is an *efficiency* point — most strength per parameter — not an engine-matching claim, and it carries ladder uncertainty (±100).

---

7. Discussion — the unifying conclusion

Across every experiment, one factor was the recurring binding constraint — sharper than "the network is the bottleneck": **the *quality of the information reaching the evaluator* (its training signal), not the loop around it** (an empirical regularity of our regime, not a theorem). The organizing law is **strength = evaluator × search**, and it explains everything once we define *information* precisely: by **new information** we mean **novel empirical data from *outside* the closed system of the net and its training set** — not a re-encoding of what is already latent. Supervision and scale inject it directly; every other method divides on one question — **does it have an external ground-truth oracle to query?**

- **Within-move search, voting, and merging do not** — they operate on fixed representations, so they only *extract* information already present (cut variance, sharpen, filter), never create what is absent.
- **Self-play and evolution can — but only through the environment.** In a **closed-form, perfect-information** game the *rules are a perfect external oracle*: search queries terminal outcomes outside any dataset — exactly how AlphaZero/Leela inject information no human game contains and surpass human play. This is a property of the *environment*: in an **open-ended, semantic** domain (language) with no external

verifier, the identical loop has nothing to query and collapses into hallucination — it can only redistribute.
- **Our chess result sits in the oracle-rich regime and still plateaued** — because at two-Mac-Studio compute the search was too weak to extract much from that oracle, *not* because it was absent. A **scale** limit, not evidence against self-play. *Mechanistically*, the escape is search depth: at AlphaZero scale a **massive per-move search budget** turns the game rules into a strong enough oracle to systematically discover deep tactical/strategic truths, generate targets that *exceed* the current net, and encode them by training — a self-reinforcing loop that breaks the weak-search/draw-collapse plateau. Our budget produces self-targets no better than the net that made them, so the loop has nothing to climb; the missing ingredient is **oracle-extraction compute**, not a different algorithm.

None of this makes redistribution *useless*: variance reduction, sharpening, and filtering are how you **reach** a ceiling cheaply — indispensable engineering. The narrow claim is about the *absolute* ceiling: **only information from outside the closed system can raise it.**

- **Architecture > parameters.** The right prior (spatial locality + weight sharing) beats raw width — a 14 MB conv reaches strength a 3× larger MLP cannot.
- **Search > scale, but search cannot exceed the net.** MCTS bought +650 Elo (2150→2800) at zero extra parameters and out-scaled fixed depth — yet flat MCTS and the cascade hit the *same* wall on the same net, because search cuts the net's *variance* (averaging noisy calls), not its *bias* (systematic blind spots). The cascade's win was **efficiency (up to 4.8× cheaper), not strength.**
- **No self-generated signal crosses the wall — we tried three.** Gradient self-play, a self-referential ladder, and evolution (given the fairest shot; its +47-Elo "escape" was noise that reversed to −30) all plateau. The same net reaches ~2150 under supervised labels, so the wall is **not capacity** but the ceiling of a system's signal about *itself* — a **scale** statement (at AlphaZero-scale compute, self-play bootstraps far past this).
- **A better *evaluator* is the only lever — and harder than it looks.** The committee's agreement predicts correctness, but **plurality voting never clearly beats the best member** (correlated blocs dominate; balance beats count); the per-position oracle is **2–3× better than the vote**, so the information is there but needs a **better aggregator** (soft-averaging / confidence-routing), not more agents.
- **Control-theory unification.** Open loop = feedforward; closed loop = MPC; self-play = iterative learning control — and in all three the learned *evaluator* caps performance.

7.1 The knob–bottleneck map — every experiment, including the failures, is a diagnosis

Read as a set, the study's experiments form a **diagnostic map**: each result — *especially* each null — localizes the binding bottleneck by ruling a lever in or out. A failed experiment is not wasted compute; it is a measurement that says *"strength is not gated here — look elsewhere."*

Knob turned	Result	Diagnosis → what binds	Move it implies
Architecture (conv vs MLP)	large gain	bias-bound — wrong prior caps a big net	fix the inductive bias *before* scaling
Capacity (2× params @ 50M)	~0 (null)	*not* capacity-bound in this data regime	add data, not parameters (here)
Data (10 → 79 shards)	+~90	data/signal-bound	more, more-diverse labels
Search amount (open → 12800 sims)	+286, then ~+55/doubling (log-linear)	search extracts value; evaluator caps its *return*	keep searching; raise the evaluator to lift the ceiling
Search allocation (cascade shape)	flat (±noise)	*not* allocation-bound at fixed budget	reallocate for **speed**, not strength
Search implementation (batch-1)	latency only	throughput-bound by engineering	batch leaves → 13–37× speed, same strength
Self-play signal	plateau	self-signal-quality-bound	can't exceed its own signal at this scale
Selection pressure (evolution)	phantom, reversed	*measurement-noise*-bound (fake gain)	re-measure cleanly before believing
Aggregation (voting 3–9 agents)	no gain	correlated-error-bound	need *diversity*, not more voters
Aggregation (ensemble-eval, merging)	no gain	combining ≠ creating knowledge	extracts existing signal, creates none
Self-distillation (fixed set)	dropped	data-*diversity*-bound (overfit)	many diverse positions, not repetition

Three things fall out:

1. Nulls are the most information-dense results. A knob that moves nothing *localizes* the bottleneck away from itself — the parameter null said "capacity isn't binding (here)," the cascade flat-line "allocation isn't," the voting sweep "more agents isn't." The one trap is **mistaking noise for signal** (evolution's phantom +47 that reversed to −30), so every promising delta was re-measured.

2. The binding lever *moves* as you relieve it. Strength is a *chain*: bias- then capacity-bound (open-loop) → search-bound → evaluator-bound (by its **data**, not parameter count).

You cannot skip a link — parameters into a data-bound net, or sims into a saturated search, buy almost nothing.

3. The map is the roadmap. The diagnosis *is* the next move: open-loop capped ⇒ add search; search saturated ⇒ a better *evaluator* (more/better data); self-signal plateaued ⇒ a better *signal source* (external labels, or AlphaZero-scale self-play). **Identify the binding lever, relieve exactly that, re-measure, repeat** — the loop that transfers to a genuinely new domain with no engine or dataset to imitate.

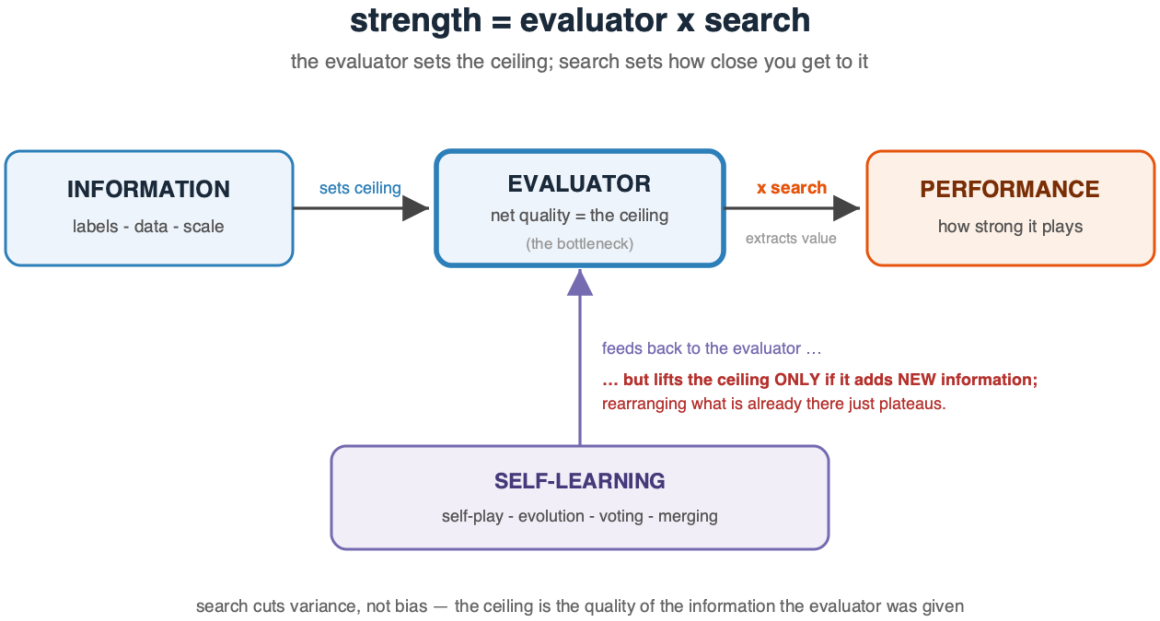
---

8. Limitations and Future Work

- Absolute Elo carries systematic uncertainty near the ladder top; relative same-ladder gains are the robust claims.
- Stage 3 is compute-limited (two machines); results characterize the *small-scale* regime.
- Stage-3 fitness/aggregation is noise-limited at our scale: relative-fitness selection with small, stochastic samples produced a **phantom** evolution "escape" that reversed under a 400-game re-match, and plurality de-biasing gaps sit inside the ~±3 CPL measurement noise. Larger, lower-variance evaluation is needed to resolve small effects.
- **Next levers, in priority order:** (1) a **better value function** (the true ceiling) via more/better search-labeled data or large-scale self-play; (2) a **better ensemble aggregator** — soft-averaging and confidence-weighted *routing* (per-position oracle 2–3× above plurality) with *balanced* cross-family diversity, not more agents; (3) **batched/parallel MCTS** and transposition tables for throughput; (4) endgame tablebases and tuned `c_puct`. All converge on one place: **improve the evaluation, and both the closed-loop and self-play ceilings rise together.**

---

9. Conclusion



This paper treated playing strength as an **efficient-allocation problem**: given a fixed budget of parameters, data, search, and latency, how do you spend it for the most capability? Measured resource by resource, the efficient mix is rarely "more parameters" — a 14 MB net reaches **~2800 Elo by *thinking (search), not growing*** — the same capability, a far cheaper mix. This is *efficiency*, not a denial of scale: the first full-data capacity point already nudges up (1× 2734 → 1.4× 2794), and **if the 2×/4× points keep climbing, parameters become co-dominant once data-starvation is relieved.** Adaptive MCTS out-scales fixed-depth search; the cascade matches it at 4.8× less compute (strength bought back as *latency*, not parameters). Self-learning is honestly negative: self-play, a self-referential ladder, and evolution all **fail to cross the ~2000 plateau** (evolution's escape was noise), and plurality committees don't reliably de-bias — though **agreement is a robust teacher-free confidence signal**.

Above all, the **method** transfers: at each stage a *single lever binds* and only an experiment reveals which. Open-loop was **capacity-bound**; adding search made us **search-bound** (search paying **log-linearly, ~+55 Elo/doubling, unsaturated through 12800**); then the evaluator binds — and at our data scale by its **data**, not its parameter count (1.4× more params bought **~0 Elo**; a full 1×/1.4×/2×/4× sweep is running). The recurring constraint, from every direction we pushed, was **the quality of the information reaching the evaluator** (§7): supervision, data, and search help; voting and merging only reorganize what the net already encodes; self-play and evolution *can* inject information, but only through an **external oracle**, so our plateau is a **compute-scale limit, not evidence that self-play fails** (AlphaZero/Leela break past human play with far more of it). What transfers is a quantified recipe and an explicit **diagnostic** for finding the binding lever.

Beyond chess. The decomposition — **capacity, inference-time search, self-generated information** — is domain-agnostic: the same three levers and the evaluator-vs-compute trade-off recur in planning, robotics, scheduling, compiler optimization, and scientific design, each with an evaluator, a decision-time search budget, and a self-improvement loop capped by its own signal quality. We measured none of those domains — only that the *decomposition* and its diagnostic (find the binding lever before investing) are what transfer, and likely outlast the chess numbers.

The clearest current echo is in large language models. The 2024–25 shift to **inference-time compute** — o1/o3, DeepSeek-R1, reasoning models — is this thesis at frontier scale: a fixed base model made far stronger by *searching over reasoning at decision time* (sampling, self-consistency, tree-of-thought) rather than by adding parameters. The mapping is direct: our *evaluator* ↔ their **base model/verifier**, our *search* ↔ their **reasoning budget**, our *self-play* ↔ their **STaR-style self-training** — with the same limit: search and self-training **convert what the model already latently knows into better answers; they cannot inject knowledge it never learned.** That is exactly why reasoning models pair search with **external verifiers or ground-truth reward** (code that runs, math that checks, tools that return facts) — the oracle that lets the loop add information. A model with no such oracle should, by our framework, **plateau**.

A test for what comes next. The framework is a **predictive filter**: pre-screen any future *synthetic-data breakthrough* or *recursively self-improving architecture* with one question — **does it inject information from outside its closed system** (new empirical data, supervision, or an external oracle: a verifier, a simulator, the physical world)? If yes, it can raise the ceiling; if it only re-processes what its own models contain, our results predict it will **plateau**. Recursive self-improvement compounds where an external oracle exists (a game's rules, a theorem checker, a compiler, a market) and stalls where none does — a claim about the *source* of information, not the method's ingenuity. - `chessnet/model.py` — conv/MLP/dual-path + value head. `chessnet/search.py` — alpha-beta, MCTS/PUCT, quiescence, the wide→narrow cascade (`MultiStageMCTSPlayer`). `chessnet/committee.py` — ensemble inference + agreement signal. `chessnet/train.py` — soft/hard + value training. `scripts/selfplay.py` — self-play iteration. - **Best model:** `runs/conv_value_llm1` (conv-96×8 + value, 3.45M params). - **Key hyperparameters:** conv width 96, depth 8; lr 5e-4 (train) / 1e-4 (self-play); gradient clip 1.0; MCTS c_puct 1.5; Dirichlet α 0.3; replay buffer 120K–300K.